

RFdiffusion HPC Installation and Execution Guide (Apptainer/SLURM)

Overview

This document outlines the standard operating procedure for pulling, configuring, and executing the RosettaCommons RFdiffusion container on a High-Performance Computing (HPC) environment using Apptainer and the SLURM workload manager.

Prerequisites

- Access to a SLURM-managed HPC cluster with NVIDIA GPUs (V100, A100, etc.).
 - Access to internet-connected nodes (or data transfer nodes) for pulling images.
 - Sufficient quota in your `scratch` or `project` space (the image requires ~6GB).
-

Phase 1: Environment Setup & Pulling the Container

By default, Apptainer caches and builds images in the user's home directory (`~/.apptainer/cache`) and `/tmp`. Because RFdiffusion is a massive image (~6GB), this often causes "No space left on device" errors due to strict home directory quotas.

1. Redirect Apptainer Cache

Before pulling the container, redirect the cache and temporary directories to your high-capacity scratch space.

```
# Create dedicated Apptainer build directories in your scratch space
mkdir -p /global/scratch/$USER/apptainer_cache
mkdir -p /global/scratch/$USER/apptainer_tmp

# Export the environment variables to point to the new directories
export APPTAINER_CACHEDIR="/global/scratch/$USER/apptainer_cache"
export APPTAINER_TMPDIR="/global/scratch/$USER/apptainer_tmp"
```

(Tip: Add the `export` lines to your `~/.bashrc` to make this persistent across login sessions).

2. Pull the Docker Image

Load the Apptainer module and pull the official RosettaCommons Docker image into a Singularity Image Format (`.sif`) file. Navigate to the directory where you want to store your containers (e.g., `/global/scratch/$USER/protein/container/`).

```
module load StdEnv/2023 apptainer/1.4.5
apptainer pull rfdiffusion.sif docker://rosettacommons/rfdiffusion:latest
```

Phase 2: Project Workspace Preparation

Create a dedicated workspace for your RFdiffusion runs. Apptainer requires absolute paths to bind directories from the host HPC into the container.

```
# Navigate to your project directory
cd /global/project/$USER/protein/rfdiffusion_test

# Create input and output directories
mkdir -p input output

# Download a sample structure for testing (Motif Scaffolding)
wget [https://files.rcsb.org/download/5TPN.pdb]
(https://files.rcsb.org/download/5TPN.pdb) -O input/5TPN.pdb
```

Phase 3: Executing via SLURM

RFdiffusion relies heavily on GPU acceleration. The container must be run via a SLURM batch script utilizing the `--nv` flag in Apptainer to pass the NVIDIA drivers through to the container.

Create a batch script named `run_rfd.sh`:

```
#!/usr/bin/env bash
#SBATCH --job-name=rfd_test
#SBATCH --gres=gpu:1          # 1 GPU is required; V100 or A100 are sufficient
#SBATCH --cpus-per-task=8    # CPU allocation for data loading/preprocessing
#SBATCH --mem=32G            # RAM allocation
#SBATCH --time=02:00:00      # Walltime (2 hours is generous for small
                             # proteins)
#SBATCH --output=rfd_test_%j.out # Standard output log (%j appends Job ID)
#SBATCH --error=rfd_test_%j.err  # Standard error log (%j appends Job ID)
#SBATCH --mail-type=END,FAIL     # Receive email notifications on job
                             # completion/failure

# 1. Load the Apptainer module
module load StdEnv/2023 apptainer/1.4.5

# 2. Define absolute paths
WORK_DIR="/global/project/hpcg6049/protein/rfdiffusion_test"
SIF_PATH="/global/scratch/hpc6049/protein/container/rfdiffusion.sif"

echo "Starting RFdiffusion job on $(hostname)"
echo "Working directory: ${WORK_DIR}"

# 3. Execute the container
# --nv: Enables NVIDIA GPU support
# --bind: Mounts host directories to the container's internal filesystem
apptainer run --nv \
  --bind ${WORK_DIR}/input:/input \
```

```
--bind ${WORK_DIR}/output:/output \  
${SIF_PATH} \  
inference.output_prefix=/output/motifscaffolding \  
inference.model_directory_path=/app/RFdiffusion/models \  
inference.input_pdb=/input/5TPN.pdb \  
inference.num_designs=3 \  
contigmap.contigs='[10-40/A163-181/10-40]'  
  
echo "Job completed successfully."
```

Submit the job to the cluster:

```
sbatch run_rfd.sh
```

Phase 4: Output Verification

Upon successful completion, check the **output/** directory. For every generated design, RFdiffusion produces two files:

1. **.pdb file:** The 3D coordinates of the generated protein backbone.
2. **.trb file:** A serialized dictionary containing essential run metadata, including evaluation metrics, loss trajectories, and residue mappings (which residues in the output correspond to the input motif).